

	Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores Computação Distribuída (Época Normal) MEIC, MEIM	
Nº	Nome:	04/01/2024

EXAME ÉPOCA NORMAL (1ª PARTE, SEM CONSULTA, DURAÇÃO: 45 min., 11 valores)

Nas questões de 1 a 5 assinale as afirmações como verdadeira (V), falsa (F) ou não responde (—). Uma opção assinalada corretamente soma 0,25 valores e incorretamente desconta 0,125 valores (50% da cotação de cada alínea).

1. [1 val.] Sobre as características dos sistemas distribuídos

☐	Para obter uma disponibilidade perto dos desejados "Five nines" (99.999%) é necessário garantir recuperação de falhas de forma automática, isto é, sem intervenção humana
☐	A escalabilidade horizontal (<i>scaling out</i>) implica ter múltiplos servidores sem estado (<i>stateless</i>) permitindo que os múltiplos pedidos dos múltiplos clientes sejam processados em servidores diferentes
☐	A escalabilidade vertical (<i>scaling up</i>) implica ter múltiplos servidores com estado (<i>stateful</i>) permitindo que os múltiplos pedidos dos múltiplos clientes sejam processados em servidores diferentes
☐	Latência (<i>Latency</i>) e Largura de banda (<i>Bandwidth</i>) são dois conceitos idênticos que se referem à velocidade de comunicação entre dois pontos de um sistema distribuído

2. [1 val.] Sobre o sistema rabbitMQ e considerando um exchange do tipo TOPIC e nome ExT

☐	Uma <i>queue</i> pode ter várias <i>binding keys</i> , por exemplo, ISEL.* e PT.ISEL.#
☐	Uma <i>queue</i> associada ao <i>exchange</i> ExT com a <i>binding key</i> ISEL.# aceita mensagens com as <i>routing key</i> ISEL.DEETC.MEIC e ISEL.DEETC.MEIM
☐	O <i>exchange</i> ExT pode ter associadas várias <i>queues</i> , em que cada uma pode ter vários consumidores
☐	Um consumidor só pode receber mensagens de uma das <i>queues</i> associadas ao <i>exchange</i> ExT

3. [1 val.] Na operação gRPC definida por `rpc oper(MsgX) returns (MsgY)` com as mensagens definidas como message MsgX { int32 x = 1; } e message MsgY { int32 y = 1; }

☐	No cliente a chamada à operação pode ser feita usando qualquer tipo de <i>stub</i>
☐	O formato da mensagem MsgX teria de ser modificado caso a operação passasse a ser <i>stream</i> de cliente
☐	No servidor a implementação da operação determina se o cliente pode ou não usar um <i>stub</i> bloqueante
☐	No servidor, a implementação da operação <code>oper</code> tem a assinatura: <code>public MsgY oper(MsgX) { . . . }</code>

4. [1 val.] Considere que existe no repositório DockerHub uma imagem de nome cd/exameApp:v1

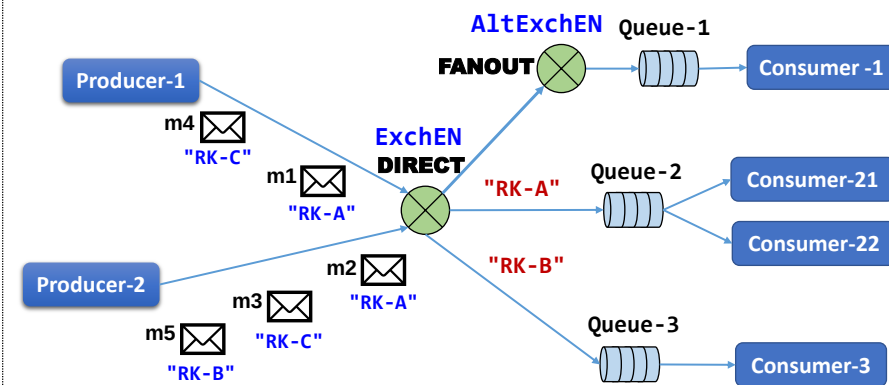
☐	Com o comando PULL pode obter-se a imagem e alterar o seu conteúdo com novas camadas
☐	É possível usar a imagem como camada base para criar (<i>build</i>) uma nova imagem
☐	Se a imagem contiver uma aplicação servidora em escuta no porto 7500 é possível executar um <i>container</i> na máquina <i>host</i> usando o porto 8000 para chamar a aplicação
☐	Se a aplicação da imagem processar ficheiros existentes na diretoria <code>/user/files</code> não é possível executar um <i>container</i> para processar ficheiros existentes na diretoria <code>/user/local/myfiles</code> da máquina <i>host</i> .

5. [1 val.] Sobre algoritmos de exclusão mútua, eleição e consensos

☐	No algoritmo <i>Raft</i> se um servidor não receber a notificação (<i>heartbeat</i>) do <i>Leader</i> corrente, durante um período de tempo, assume que não há <i>Leader</i> e inicia uma eleição para se obter um novo <i>Leader</i>
☐	O algoritmo de exclusão mútua centralizado é simples de implementar, mas ao ter um servidor para gerir o <i>token</i> de acesso à região crítica tem limitações de escalabilidade e de tolerância a falhas
☐	O conceito de <i>Group</i> e de notificações de <i>membership</i> do <i>middleware</i> Spread, permite implementar um algoritmo de eleição muito simples e sem troca de mensagens entre os membros do grupo, desde que se assuma que o <i>Leader</i> ou coordenador é sempre o primeiro a entrar (<i>join</i>) no grupo
☐	O algoritmo <i>Ricart&Agrawala</i> permite eleger como <i>Leader</i> o participante com menor relógio lógico

6. [2 val.] Considere um sistema distribuído, constituído por 3 participantes (**p0**, **p1**, **p2**), que utiliza comunicação por *multicast* fiável e um mecanismo baseado em relógios vetoriais que garante ordenação causal de mensagens. Quando **p0 envia a mensagem** com o vetor **[5,5,5]**, justifique o que acontece na receção da mensagem nos participantes p1 e p2 e como ficam os relógios locais, considerando:
- p1** tem o seu vetor local **p1=[4,7,6]**;
 - p2** tem o seu vetor local **p2=[4,4,7]**;

7. [2 val.] Considere um sistema desenvolvido com o middleware RabbitMQ que permite aos produtores publicar mensagens no exchange **ExchEN** do tipo **DIRECT** que tem associado o *Alternate Exchange* **AltExchEN** do tipo **FANOUT**. Considerando as routing keys ("RK-A", "RK-B", "RK-C") indicadas nas mensagens m1, m2, m3, m4 e m5, publicadas pelos dois *producers* e as associações das várias *Queues*, escreva sobre o diagrama da figura o fluxo de encaminhamento dessas mensagens, indicando claramente à frente de cada consumidor as mensagens que cada um pode receber.



8. [2 val.] Considere que existe uma imagem Docker com uma aplicação gRPC no porto 9000 para processar imagens jpg, que aceita por cada pedido um nome de um ficheiro, existente na diretoria **/usr/local/**, criando na mesma diretoria uma nova imagem num ficheiro com o nome da imagem inicial conjugado com o sufixo "proc". Infelizmente na máquina *host*, onde se pretende executar múltiplos *containers*, o porto 9000 já está ocupado por outro serviço. Descreva como pode executar múltiplos *containers* na máquina *host* para processar múltiplas imagens existentes na diretoria **/usr/fotos/**.